AA411

Fault Isolator Test Procedure

Team Name: ROFASO

Project Name: Damage Assessment Protocol (D.A.P.)

Sub-system Name: Fault Isolator

Written By: Josh Rolfe, Oleksiy Polyakov, Mak Sukimoto

Date: 5/12/2026

Test Dates: 05/15/2026

Test Team:

Name	Initials
Mak Sukimoto	MS
Josh Rolfe	JR
Holland Kantner	HK
Oleksiy Polyakov	OP

Contents

1 Objective of Test Procedure **2**

2 Measurements **2**

3 Equipment Required **2**

4 Test Procedure **2**

i Test Case ID: `TC-FI-01` — Fault Isolation Validation 2

1 Objective of Test Procedure

The goal of this procedure is to test that the fault isolator can accurately diagnose wing, rudder, or no damage to the Seaglider and output a best match.

The following are the requirements that will be tested using this procedure.

- **F-I1:** The Fault Isolator shall identify the lowest percent difference between single 0-100% wing or rudder DT cases and mission data.

2 Measurements

Variable	Description	Units	Measurement Method
MAPE	mean absolute percent error	All	Percent Difference

3 Equipment Required

Equipment/Tools	Description	Qty	Check
Laptop	At least 8 GB RAM and 256 GB SSD is required	1	
MATLAB R2024a	Simulation environment. Add ons needed: • Aerospace Toolbox • Control System Toolbox • Optimization Toolbox • Robust Control Toolbox	1	
Test Cases in a Matrix	Input dataset	1	
Final Fault Isolator Code	Code meant to determine whether a fault has occurred	1	
Seaglider Data	Mission Data	1	

4 Test Procedure

i Test Case ID: `TC-FI-01` — Fault Isolation Validation

1. **Environment Initialization**
   Open MATLAB and ensure the working directory contains `D_fault_isolator.m`, `missioncompare_version_holland_4.m`, and the required `.eng` data files.

2. **Data Ingestion and Pre-processing**
   Execute the following example commands to load the mission data and digital twin library:

```
%% Dive 6
MI_Data = missioncompare_unpack_eng_to_dive_data('p1940006.eng');
```

Listing 1: Data Ingestion Script

3. **Comparator Execution**
   Run the comparator version 4 to generate the diagnostic structures for each library case:

```
%% 4. Run the Pipeline
fprintf('--- Running Comparator for all cases ---\n');
out_Perfect = missionIsolate_version_holland_4(DT_Perfect, MI_Data);
out_Small   = missionIsolate_version_holland_4(DT_SmallError, MI_Data);
out_Large   = missionIsolate_version_holland_4(DT_LargeError, MI_Data);
out_Zeros   = missionIsolate_version_holland_4(DT_WithZeros, MI_Data
```

Listing 2: Comparator Execution

4. **Fault Isolator Ranking**
   Execute the isolator to perform the final tabulation and ranking:

```
[RankingTable, BestMatch] = D_fault_isolator(out_Large, out_Perfect,
    out_Small, out_Zeros);
```

Listing 3: Isolator Execution

5. **Results Verification**
   Inspect the generated `RankedTable` and verify the following criteria:

Verification Point	**Expected Result**	**Pass/Fail**
Table Generation	`RankedTable` exists in workspace as a 2-column table.	
Ranking Order	`Percent_Match_Error` is sorted in ascending order (lowest percent difference to highest percent difference)	
Diagnosis Accuracy	`BestMatch` matches the known fault case.	

6. **Data Collection**
   Save data and upload to Oceans Google Drive.
   (a) AA Resilient Ocean Flight Autonomy for Seaglider Operation → Test Results → Fault Isolator

Shut Down

1. Save *D_fault _isolator.m*
   (a) Saved? Yes/No ________________
2. Close MatLab
   (a) MatLab Closed? Yes/No ________________
3. Shut down laptop
   (a) Power Off? Yes/No ________________